

Computational Modeling for Psychology Students

Britt Anderson

Winter 2026

Contents

1	Welcome to Psych 420	1
1.1	Introductions	1
1.2	What is computational cognitive (neuro)science?	2
1.3	Syllabus Review	2
1.3.1	Course number and title	2
1.3.2	Term and year of offering	2
1.3.3	Class days, times, building, and room number	2
1.3.4	Class instructor's name, office, contact information, office hours	2
1.3.5	Teaching assistant's name, office, contact information, office hours (if applicable)	2
1.3.6	Course description	2
1.3.7	Course objectives	3
1.3.8	Required text and/or readings	3
1.3.9	A general overview of the topics to be covered	3
1.3.10	The evaluation structure for the course including course requirements, deadlines, weight of requirements to- ward the final course grade	3
1.3.11	Acceptable rules for group work (See Group Assign- ment Checklist)	4
1.3.12	Indication of how late submission of assignments and missed assignments will be treated	5
1.3.13	Indication of where students are to submit assign- ments and pick up marked assignments	5
1.3.14	Intellectual Property	5
1.3.15	Chosen/Preferred First Name	6
1.3.16	Mental Health Support	6

1.3.17	Territorial Acknowledgement	7
1.3.18	Policy 33, Ethical Behaviour	7
1.4	A look ahead – what we will code	7
1.5	A look ahead – what we talk about	8
1.6	What is Cognition?	8
1.7	Is a mathematical formula a model?	8
1.7.1	Forgetting Curve — Ebbinghaus	8
1.7.2	A Mathematical Function	8
1.7.3	What can we infer	8
1.7.4	Generate your own plot of the exponential function.	8
2	Warming Up: Action Potentials	8
2.1	What is an action potential?	10
2.2	Notation	11
2.3	Differential Equations	11
2.4	Using a DE	12
2.5	Using DEs to compute a square root	12
2.6	Using DEs - Practicing with Springs	13
2.6.1	Now You Try with a Damped Oscillator	15
2.7	Integrate and Fire Neuron	15
2.7.1	Historical Notes	15
2.7.2	Formula I and F	16
2.7.3	Coding up the Integrate and Fire Neuron	18
2.8	Integrate and Fire Homework	21
2.9	Hodgkin and Huxley Model	22
2.9.1	Who were Hodgkin and Huxley	22
2.9.2	Model Description	22
3	What is computation and what is its role in cognitive science?	30
3.1	What do you think?	30
3.2	What do we need to know?	30
3.3	Computing in Minds and Computers	31
3.4	What is a Turing machine? Some Background and Details	31
3.4.1	Classroom Activity	32
3.5	The Lambda Calculus	32
3.5.1	Some Details and Interesting Facts about the Lambda Calculus	32
3.5.2	Into the Weeds	32
3.5.3	Some Opportunities for Practice	33

3.5.4	What are the differences?	34
3.6	Classical Computational Theory of Mind	34
3.7	Programming a Turing Machine: the Busy Beaver	34
3.7.1	A Busy Beaver Warm-Up	36
3.7.2	Busy Beaver Problem Demo Code	36
4	Theory	38
5	Math and Code	38

1 Welcome to Psych 420

1.1 Introductions

1. Who am I?
2. Who are you? Send me an email (britt@uwaterloo.ca) right now (from your uwaterloo address) with
 - (a) Your name (and any pronunciation tips).
 - (b) Area of concentration,
 - (c) Year of study,
 - (d) Programming language you are most capable in (and your level of competency),
 - (e) Reason you took this course.
3. What you will need: A functioning computer that you know how to use. What to do now:
 - Update your operating system.
 - Verify you know how to find a file on your computer (e.g. do you know the name of your home directory)?
 - Confirm you know how to install software on your computer.
 - You know (or will know by next week) what `git` is and how to use it, and that you will have an IDE installed.

1.2 What is computational cognitive (neuro)science?

1. Form teams of two (or three if odd number).
2. Go to the following web site: 2025 Posters Cognitive Computational Neuroscience meeting.
 - (a) Pick a poster. Note it's label (letter-number; on the left)
 - (b) Be ready with a two sentence summary.
 - (c) Tell us why it is a good example of computational modelling of either mind or brain.
 - (d) Why you chose it.

1.3 Syllabus Review

1.3.1 Course number and title

PSYCH420 Introduction to Computational Neuroscience Methods

1.3.2 Term and year of offering

Winter 2026

1.3.3 Class days, times, building, and room number

Th 11:30 - 14:20 HH 2107

1.3.4 Class instructor's name, office, contact information, office hours

Britt Anderson, PAS 4032, x 43056, Office hours by arrangement

1.3.5 Teaching assistant's name, office, contact information, office hours (if applicable)

1.3.6 Course description

An introduction to the basic elements of computational modeling with an emphasis on psychology students.

1.3.7 Course objectives

1. You will understand the scope of subjects and tools that comprise computational neuroscience and computational cognitive modeling.
2. You will have a better understanding of theory development and how it relates to ideas about computation, programming, mind and brain.
3. You will have a better understanding of how programming language choice influences model and theory development.
4. You will learn some of the fundamental notation and concepts of the mathematical domains that form the basis of much computational modeling in neuroscience and psychology.
5. You will gain experience in the practical aspects of programming simple models by doing.

1.3.8 Required text and/or readings

There is no required text book. Material that might be needed outside what I present in class will be made available on line, predominantly at the course repo on github. Make sure you are tracking the **W2026** branch.

As all University students should have access to a computer, and a computer is required for this course, there is \$0 CAD additional expense.

1.3.9 A general overview of the topics to be covered

See Course objectives.

1.3.10 The evaluation structure for the course including course requirements, deadlines, weight of requirements toward the final course grade

Category	Contribution
Participation	12%.
Final Project	20%.
Homeworks	68%

It is easy to find high quality material to read or watch. What good then is the classroom? The classroom is for doing and clarifying. You have no way of knowing if your understanding developed from online material is complete or accurate unless you match it off against others. Your peers and

your instructor. Thus, we shouldn't use class time for presenting material that can be read or watched anywhere anytime, but we should use it for testing whether your knowledge is sound, flexible, and deployable. We will therefore read, discuss, and collaborate.

Learning math and computing, and how to use them, is not something that can be learned through observation. Like playing piano, or improving your golf swing, improvement comes from practice, practice, practice. It is not enough to merely state the principles. You must demonstrate that they have been internalized. We will therefore do small exercises in class. And since one of the best ways to learn is to teach, we will frequently do these exercises in small groups. You might find your peers description of a problematic fact more useful than mine since they are at a more similar learning level. You may find your own understanding grows as you have to reframe my teachings in your own words to help a fellow student grasp a difficult concept.

For these reasons class participation is important. It is where we will test our comprehension of important points, I will learn if a concept is not well appreciated, and you will have the chance to practice the skills you later may wish to use in your own way on your own problems. Each week's attendance is worth 1% of the final grade.

I like to give lots of small homeworks. That takes the pressure of any one assignment, and means you get both frequent feedback on your performance, and you are coerced into not falling behind.

The final project is meant to give you an opportunity to showcase your learning and to use the general and specific skills that you will use in the future, whether in academics or industry, for sharing complex technical material with others.

1.3.11 Acceptable rules for group work (See Group Assignment Checklist)

Group work is the norm in post-graduate life (both academic and professional). Group work will be graded as a group. All group members will get the same grade for their group project. More details on the nature of the group project will be shared during the term.

1.3.12 Indication of how late submission of assignments and missed assignments will be treated

If I know that an assignment will be late in advance, and the reason for the tardiness is beyond your reasonable control, then I probably won't take off any points for late submission. Otherwise, I may reduce the possible maximal score by 10% per week.

1.3.13 Indication of where students are to submit assignments and pick up marked assignments

Assignments will be submitted in dropboxes on Learn. Submissions and Announcements are the main uses we will make of learn. All other material will be shared in class or via the github repository.

1.3.14 Intellectual Property

Intellectual property includes items such as:

- Lecture content, spoken and written (and any audio/video recording thereof);
- Lecture handouts, presentations, and other materials prepared for the course (e.g., PowerPoint slides);
- Questions or solution sets from various types of assessments (e.g., assignments, quizzes, tests, final exams); and
- Work protected by copyright (e.g., any work authored by the instructor or TA or used by the instructor or TA with permission of the copyright owner).

Course materials and the intellectual property contained therein, are used to enhance a student's educational experience. However, sharing this intellectual property without the intellectual property owner's permission is a violation of intellectual property rights. For this reason, it is necessary to ask the instructor, TA and/or the University of Waterloo for permission before uploading and sharing the intellectual property of others online (e.g., to an online repository).

Permission from an instructor, TA or the University is also necessary before sharing the intellectual property of others from completed courses with students taking the same/similar courses in subsequent terms/years. In many cases, instructors might be happy to allow distribution of certain

materials. However, doing so without expressed permission is considered a violation of intellectual property rights.

Please alert the instructor if you become aware of intellectual property belonging to others (past or present) circulating, either through the student body or online. The intellectual property rights owner deserves to know (and may have already given their consent).

1.3.15 Chosen/Preferred First Name

Do you want professors and interviewers to call you by a different first name? Take a minute now to verify or tell us your chosen/preferred first name by logging into WatIAM.

Why? Starting in winter 2020, your chosen/preferred first name listed in WatIAM will be used broadly across campus (e.g., LEARN, Quest, WaterlooWorks, WatCard, etc). Note: Your legal first name will always be used on certain official documents. For more details, visit Updating Personal Information.

Important notes

If you included a preferred name on your OUAC application, it will be used as your chosen/preferred name unless you make a change now. If you don't provide a chosen/preferred name, your legal first name will continue to be used.

1.3.16 Mental Health Support

All of us need a support system. The faculty and staff in Arts encourage students to seek out mental health support if they are needed.

1. On Campus

Counselling Services: counselling.services@uwaterloo.ca / 519-888-4567 ext. 32655 MATES: one-to-one peer support program offered by the Waterloo Undergraduate Student Association (WUSA) and Counselling Services

2. Off campus, 24/7

Good2Talk: Free confidential help line for post-secondary students. Phone: 1-866-925-5454 Grand River Hospital: Emergency care for mental health crisis. Phone: 519-749-4300 ext. 6880 Here 24/7: Mental Health and Crisis Service Team. Phone: 1-844-437-3247 OK2BME: set of support services for lesbian, gay, bisexual, transgender or questioning teens in Waterloo. Phone: 519-884-0000 extension 213

1.3.17 Territorial Acknowledgement

Academic freedom at the University of Waterloo We acknowledge that we are living and working on the traditional territory of the Attawandaron (also known as Neutral), Anishinaabe and Haudenosaunee peoples. The University of Waterloo is situated on the Haldimand Tract, the land promised to the Six Nations that includes ten kilometres on each side of the Grand River.

For more information about the purpose of territorial acknowledgements, please see the CAUT Guide to Acknowledging Traditional Territory. Academic freedom at the University of Waterloo

1.3.18 Policy 33, Ethical Behaviour

Policy 33, Ethical Behaviour states, as one of its general principles (Section 1), “The University supports academic freedom for all members of the University community. Academic freedom carries with it the duty to use that freedom in a manner consistent with the scholarly obligation to base teaching and research on an honest and ethical quest for knowledge. In the context of this policy, ‘academic freedom’ refers to academic activities, including teaching and scholarship, as is articulated in the principles set out in the Memorandum of Agreement between the FAUW and the University of Waterloo, 1998 (Article 6). The academic environment which fosters free debate may from time to time include the presentation or discussion of unpopular opinions or controversial material. Such material shall be dealt with as openly, respectfully and sensitively as possible.” This definition is repeated in Policies 70 and 71, and in the Memorandum of Agreement, Section 6

1.4 A look ahead – what we will code

1. Turing Machine
2. Integrate and Fire Neuron
3. Hodgkin-Huxley Neuron
4. Perceptron
5. Hopfield Net
6. Agent Based Model
7. And possibly more

1.5 A look ahead – what we talk about

1. What is cognition?
2. What is a cognitive model?
3. What is the relationship between cognition and neuroscience?
4. All of which will benefit from better understanding what is a theory.

1.6 What is Cognition?

Groups of three. No two members should be the same as the last group. What is Cognition? Each group gets an opinion (want three of four groups) Brainard/Byrne/Heyes/Suddendorf 20 minutes to read and discuss to make sure they understand. Then rotate the groups so that each group has only one member/opinion. 15 minutes to explain the opinions to each other. General discussion.

1.7 Is a mathematical formula a model?

1.7.1 Forgetting Curve — Ebbinghaus

1.7.2 A Mathematical Function

```
import Graphics.Gnuplot.Simple
plotFunc [PNG "./images/expdecay.png"]
(linearScale 100 (1,6))
((\x -> exp (1.0/x)) :: Double -> Double)
```

1.7.3 What can we infer

from the similarity between Figure 1 and Figure 2 ?

1.7.4 Generate your own plot of the exponential function.

2 Warming Up: Action Potentials

Our goals:

1. Understand what we are trying to model: the action potential.
2. Learn some of the prior work.

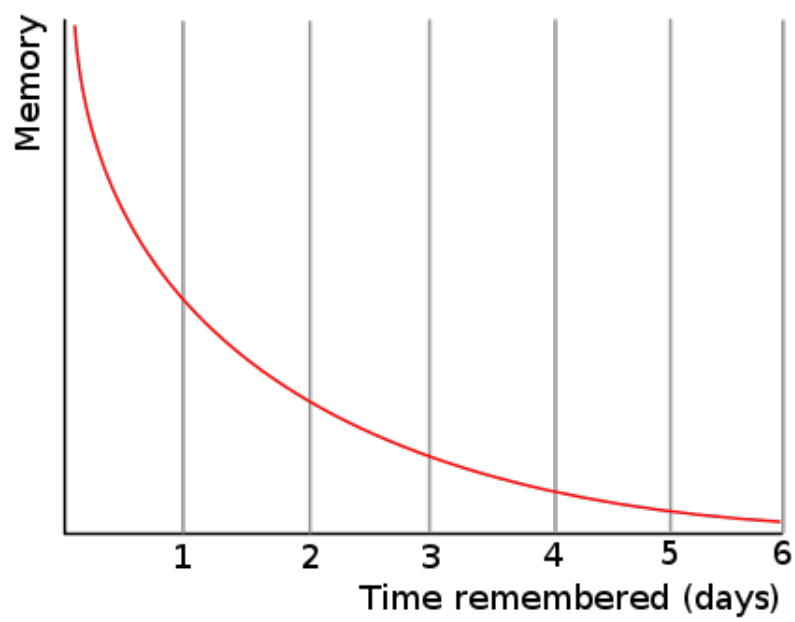


Figure 1: By The original uploader was Icez at English Wikipedia.

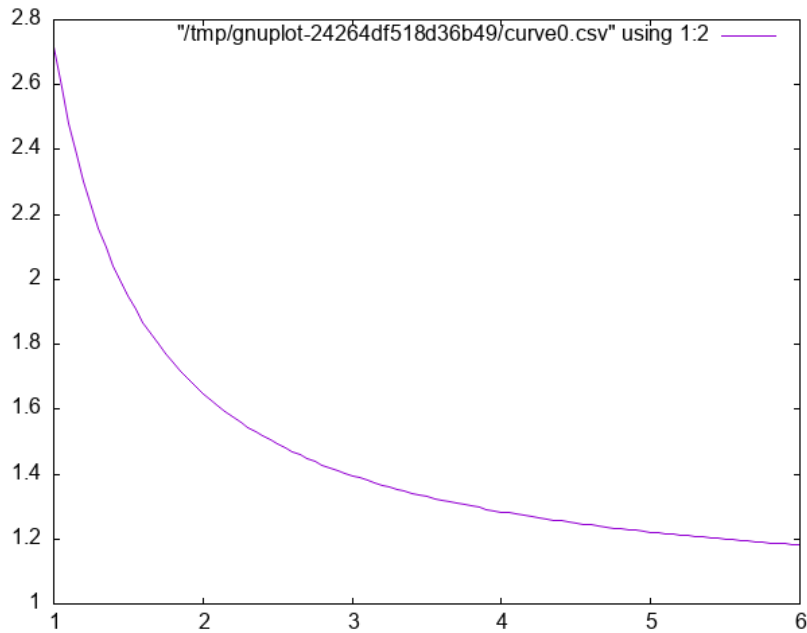


Figure 2: Plot of $e^{\frac{1}{x}}$.

3. Understand the mathematical background for an action potential.
 - (a) Simple electronics.
 - (b) Simple differential equations.
4. Make ourselves familiar with the basic notation of these two background areas.
5. Implement simple computational versions of spiking neurons. If you cannot code it, you do not understand it.

2.1 What is an action potential?

1. Draw one.
2. Explain what ions account for the different parts of the curve.
3. Why is an action potential like a flush toilet?
4. Why did/do people use the action potential to argue that brains are like digital computers?

5. Who were McCulloch and Pitts and why are they relevant?
6. Who developed the first detailed biophysical model of the action potential and what became of them?
7. Why would we want to write a program to simulate a spiking neuron?
8. Why might we want a simpler version?
9. What maths are important for modeling spiking neurons?

2.2 Notation

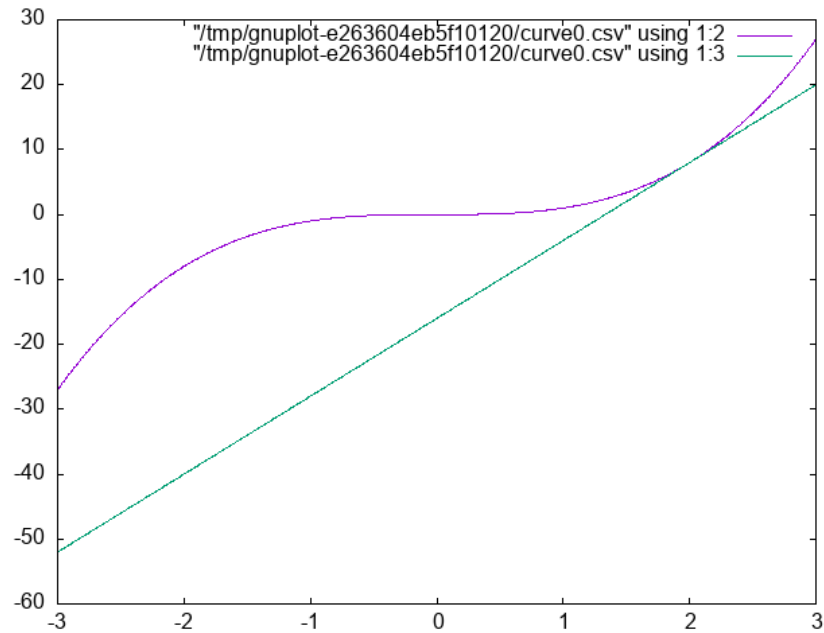
The mathematician's interest in being concise often means that simple ideas get translated to unfamiliar symbols. \ Just because the notation is unfamiliar does not mean the idea is complicated. What does $\sum_{x \in \{1,2,3\}} x = 6$ mean?

Different mathematical and computational traditions have their own preferences, thus the same idea can have different notations. To a mathematician the $\sqrt{-1} = i$. To the engineer it is j . $\frac{dx}{dt}$ $\dot{x}$ x' $f'(x)$ are all different ways of talking about derivatives.

2.3 Differential Equations

1. What is a differential equation (aka de)?
2. What is a derivative?
3. What is the **definition** of a derivative? $\frac{dx}{dy} = \frac{f(x+\Delta x) - f(x)}{x + \Delta x - x}$
4. What is a slope of a line?
5. Of a curve?
6. Provide examples of phenomena in psychology or neuroscience (other than the action potential) that might be good candidates for this mathematical technique.

```
plotFuncs [PNG "./images/derivExample.png"] (linearScale 1000 (-
3 , 3 ) )
[(\x -> x ** 3) :: Double -> Double
 , (\x -> 12 * x - 16) :: Double -> Double ]
```



2.4 Using a DE

In computational neuroscience (or psychology) we don't typically "solve" DEs. We use them. We use them to help us move forward in time to understand a process by virtue of what we know about its rate of change. We will see that in a couple of short examples, and then some simple scenarios for you to play with.

2.5 Using DEs to compute a square root

```

1 xcubed :: Double -> Double
2 xcubed = (** 3)
3
4 diffXCubed :: Double -> Double
5 diffXCubed = ( 3 *) . ( ** 2)
6
7 getStep :: Double -> Double -> Double
8 getStep guess goal = ( goal - xcubed guess) / diffXCubed guess
9
10 getCubeRoot :: Double -> Double -> Double -> Double
11 getCubeRoot g i t =

```

```

12   let  cg = i + getStep i g
13       myerr = abs (xcubed cg - g) in
14       if myerr > t then getCubeRoot g cg t else cg

```

Now let's test it:

```

:load "./code/sqrt.hs"
getCubeRoot 27 1 0.001
getCubeRoot 8 1 0.001
getCubeRoot 141 1 0.001

```

```

[1 of 2] Compiling Main                ( code/sqrt.hs, interpreted )
Ok, one module loaded.
3.00000005410641766
2.0000004911675504
5.204827869200383

```

2.6 Using DEs - Practicing with Springs

Our Hodgkin-Huxley model will be will require us to use several different equations. And though the intergrate and fire neuron only requires one, a neuron is still rather abstract as a computational entity. Therefore, we will begin our DE coding practice with a simple, idealized spring.

Our differential equation is $\frac{d^2 position}{dt^2} = -P position$.

where 'position' refers to the location in space of the unattached end of our spring (we assume one end is anchored to a wall or something), 't' refers to time, and 'P' is a constant, often called the spring constant, that indicates how springy the spring is.

If we knew where the spring was now, we could use this equation to tell us where it would be a little bit into the future; just as you can use your velocity on the highway to estimate your location in the future. The smaller the time step the more accurate will be our estimate. This simple Euler method often works quite well for smooth functions that don't change very much.

Unfortunately the spring equation is for the **second** derivative of location relative to time. This is called acceleration. Thus we need a two step process. From our current location and this equation estimate our velocity. From our present location and our velocity estimate our location. And repeat.

```

1  {-# LANGUAGE GADTs #-}

```

```

2  module Spring where
3
4  data SpringState where
5      SpringState :: {
6          sDt :: Double,
7          sAccFxn :: Double -> Double,
8          sVel :: Double,
9          sLoc :: Double } -> SpringState
10
11 instance Show SpringState where
12     show :: SpringState -> String
13     show s = "SpringState { t: " ++ show (sDt s) ++ "\
14               \, v: " ++ show (sVel s) ++ "\
15               \, loc: " ++ show (sLoc s) ++ " }"
16
17
18 {- Initial Values -}
19
20 initV :: Double
21 initV = 0
22 initLoc :: Double
23 initLoc = 10
24 sprConst :: Double
25 sprConst = 2.0
26 timeStep :: Double
27 timeStep = 0.05
28 dt :: Double
29 dt = timeStep
30
31
32 {- Helper Functions -}
33
34 accel :: Double -> Double -> Double
35 accel sc loc = (-sc) * loc
36
37 accWithSC :: Double -> Double
38 accWithSC = accel sprConst -- called currying
39
40 eulerUD :: Double -> Double -> Double -> Double
41 eulerUD ts roc oldval = oldval + roc * ts

```

```

42
43 springLoop :: SpringState -> SpringState
44 springLoop ss =
45     let cA = sAccFxn ss (sLoc ss)
46         cV = eulerUD (sDt ss) cA (sVel ss)
47         cL = eulerUD (sDt ss) cV (sLoc ss)
48     in ss {sVel = cV, sLoc = cL}
49
50
51 releaseSpring :: Int -> [SpringState]
52 releaseSpring maxIter =
53     take maxIter (iterate springLoop
54                     (SpringState timeStep
55                      accWithSC initV
56                      initLoc)) -- makes use of lazy evaluation

:load "./code/Spring.hs"
plotList [PNG "./images/spring.png"]
    $ map (\s -> sLoc s) $ releaseSpring 2000

```

2.6.1 Now You Try with a Damped Oscillator

The changes to whatever code you have for the frictionless spring should require very little alteration to work with the following DE:

$$\frac{d^2s}{dt^2} = -P s(t) - k v(t)$$

2.7 Integrate and Fire Neuron

Is the integrate and fire model used much in modeling in the present time? answer

2.7.1 Historical Notes

1. Louie La Pique Modern Commentary on Lapique's Neuron Model
Original Lapique Paper (translated) Brief Biographical Details of Lapicque
2. Lord Adrian
 - (a) When was the action potential demonstrated?

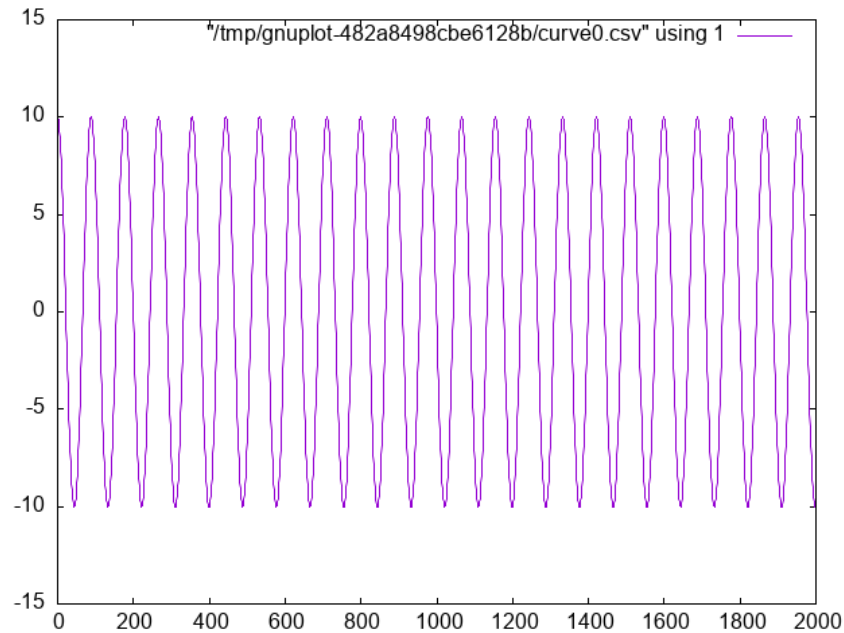


Figure 3: Frictionless Springs Oscillate

(b) What was the experimental animal used by Adrian?

<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1420429/pdf/jphysiol01990-0084.pdf>

2.7.2 Formula I and F

$$\tau \frac{dV(t)}{dt} = -V(t) + R I(t)$$

Why is this the formula?

1. Simple Electronics The following questions are the ones we need to answer to derive our integrate and fire model.

- (a) What is Ohm's Law?
- (b) What is Kirchoff's Point Rule?
- (c) What is Capacitance?
- (d) What is the relation between current and charge?

2. Deriving the IandF Equation



1437 PARIS. — La Sorbonne, laboratoire de physiologie, M. Lapicque
(Electricité)

N.D. Phot.

Figure 4: La Pique Laboratory at the Sorbonne Paris

Can you tell why, looking at the integrate and fire equation, if we don't reach the firing threshold, we see an exponential decay?

$$I = I_R + I_C \quad (a)$$

$$= I_R + C \frac{dV}{dt} \quad (b)$$

$$= \frac{V}{R} + C \frac{dV}{dt} \quad (c)$$

$$RI = V + RC \frac{dV}{dt} \quad (d)$$

$$\frac{1}{\tau}(RI - V) = \frac{dV}{dt} \quad (e)$$

- a: Kirchhoff's point rule,
- b: the relationship between current, charge, and their derivatives
- c: Ohm's law
- d: multiply through by R
- e: rearrange and define τ

2.7.3 Coding up the Integrate and Fire Neuron

Approach this with the same ideas as the spring example. You have a certain initial value, and you update that using your differential equation that relates how much voltage changes as a function of time and current voltage.

Unlike real neurons the Integrate and Fire neuron model does not have a natural threshold and spiking behavior. You pick a threshold and everyone time your voltage reaches that threshold you designate a spike and reset the voltage.

Additional Activities (if that goes too quickly):

- Add a refractory period to the Integrate and Fire model.
- Alter the input current
- Change the form of the input current

```

1 {-# LANGUAGE GADTs #-}
2
3 module IandF where
4
5 dt,maxt,initt,starttime,stoptime,cap
6   ,res,threshold,spikedisplay,initv
7   ,voltage,injectioncurrent
8   ,tau :: Double
9 injectiontime,runningTime :: [Double]
10 dt = 0.05
11 maxt = 10.0
12 initt = 0.0
13 starttime = 1.0
14 stoptime = 6.0
15 cap = 1.0
16 res = 2.0
17 threshold = 3.0
18 spikedisplay = 8.0
19 initv = 0.0
20 voltage = initv
21 injectioncurrent = 4.3
22 injectiontime = [starttime, stoptime]
23 tau = res * cap
24 runningTime = [0.0]
25
26 data IandFStrut where
27   IandFStrut :: {
28     time :: [Double],
29     spikestatus :: Bool,
30     currents :: [Double],
31     voltages :: [Double]
32   } -> IandFStrut
33   deriving Show
34
35 instance Semigroup IandFStrut where
36   (IandFStrut t1 _ c1 v1) <> (IandFStrut t2 s2 c2 v2) =
37     IandFStrut (t1 ++ t2) s2 (c1 ++ c2) (v1 ++ v2)
38
39 initialStrut :: IandFStrut
40 initialStrut = makeStep 0 False 0.0 initv

```

```

41
42 update :: Double -> Double -> Double -> Double
43 update ov roc ts = roc * ts + ov
44
45 dvdt :: Double -> Double -> Double -> Double
46 dvdt lres li lv =
47   (1 / tau) * ( lres * li - lv )
48
49 between :: Double -> Double
50 between ct = case injectiontime of
51   [first,last] ->
52     if ct >= first && ct <= last
53     then injectioncurrent
54     else 0.0
55   badList -> error "Error: Expected list of two doubles"
56
57 vchoice :: (Double, Bool, Double, Double) -> Double
58 vchoice (cv,ss,thr,sd)
59   | ss = 0.0
60   | cv > thr  && not ss = sd
61   | otherwise = cv
62
63 makeStep :: Double -> Bool -> Double -> Double -> IandFStrut
64 makeStep t s c v = IandFStrut [t] s [c] [v]
65
66 oneRunIandF :: IandFStrut -> IandFStrut
67 oneRunIandF inStrut =
68   let ct = last (time inStrut)
69       cv = last (voltages inStrut)
70       ci = between ct
71       nv = vchoice (update cv (dvdt res ci cv) dt
72                     , spikestatus inStrut
73                     , threshold, spikedisplay)
74       nextstep = makeStep (ct + dt)
75                   (abs (nv - spikedisplay) < threshold)
76                   ci
77                   nv
78   in (inStrut <> nextstep)

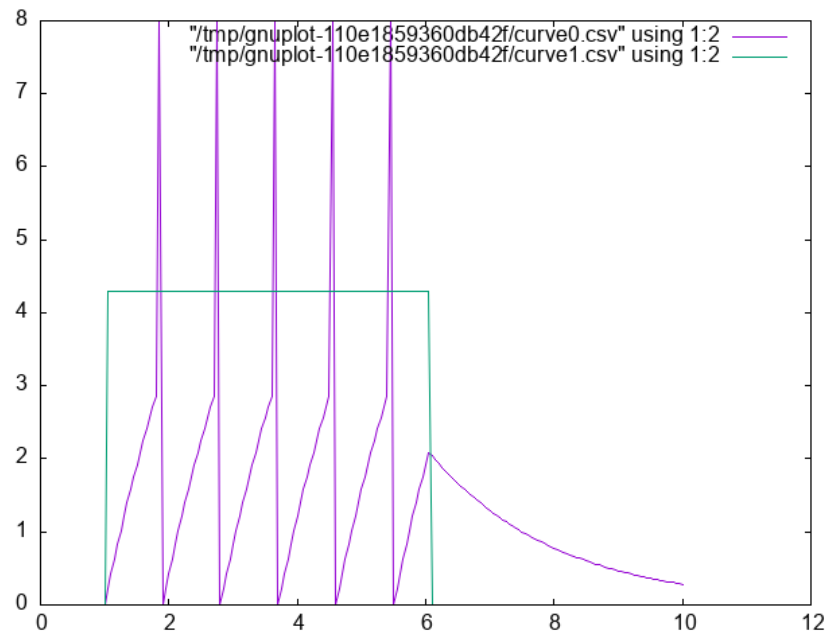
:load ./code/IandF.hs

```

```

let run200 = iterate oneRunIandF initialStrut !! 200
    vs = voltages run200
    cs = currents run200
    ts = time run200
    tv = zip ts vs
    tc = zip ts cs
plotPaths [PNG "./images/iandf.png"] [tv,tc]

```



2.8 Integrate and Fire Homework

The Integrate and Fire homework has two components. One practical and one theoretical.

Practically, submit an integrate and fire program. Do something to show that you had something to do with the coding and you understand what is happening.

Theoretically, look at this article (pdf) and tell me how you feel our integrate and fire model compares to these actual real world spiking data when both are give constant input. What are the implications for using the integrate and fire model as a model of neuronal function?

2.9 Hodgkin and Huxley Model

2.9.1 Who were Hodgkin and Huxley

The goal in this biological scavenger hunt is to try and develop some ideas about what makes one able to effectively contribute to computational neuroscience. As they undoubtedly did. I will divide you into small groups each targeting either Hodgkin or Huxley. Then we will mix up the groups for sharing. Then we will come back together to talk very briefly about who they were, but at somewhat greater length about what we learned about what sort of education or other experiences we should seek out if we want to contribute to this field.

2.9.2 Model Description

Gerstner's (free) book's chapter on the HandH model.

1. Comments and Steps in Coding the Hodgkin Huxley Model

Some introductory reminders and admonitions:

The current going in to the cell is intended to represent what an electrophysiologist would inject in their laboratory setting, or what might be changed by the input from other neurons. The total current coming out of the neuron is the sum of the capacitance (due to the lipid bilayer), and the resistance (due to the ion channels). This is **Kirchoff's** rule implemented in the Hodgkin and Huxley model.

Unlike the Integrate and Fire model, which lumps all the ionic currents together, the HandH model decomposes the conductance into three parts. You have a resistance *for each ion channel*. The rule for currents in parallel is to apply Kirchoff's and Ohm's laws realizing that they all experience the same voltage, thus the currents sum. The Hodgkin and Huxley model has components for Sodium (Na), Potassium (K), and negative anions (still lumped as "leak" l).

$$\sum_i I_R(t) = \bar{g}_{Na} m^3 h l (V(t) - E_{Na}) + \bar{g}_K n^4 (V(t) - E_K) + \bar{g}_L (V(t) - E_L)$$

$$I_{tot} = I_r + I_C$$

$$I_{tot} = \bar{g}_{Na} m^3 h (V(t) - E_{Na}) + \bar{g}_K n^4 (V(t) - E_K) + \bar{g}_L (V(t) - E_L) + c \frac{dV}{dt}$$

If you rearrange terms you can get the $\frac{dV}{dt}$ on one side of the equation by itself.

$$c \frac{dV}{dt} = I_{tot} - (\bar{g}_{Na} m^3 h (V(t) - E_{Na}) + \bar{g}_K n^4 (V(t) - E_K) + \bar{g}_L (V(t) - E_L))$$

You cannot program what you don't understand. Don't start writing code too soon. Writing code won't make a poorly understood idea clearer. Though it can reveal that you don't understand what you thought you understood. It is often best to start sketching ideas on paper or a blackboard before you open up your IDE.

For instance,

- What are the $\bar{g}_*$ terms?
- What are the E_* terms?
- What do m, n, and h represent?
- Where did these equations come from?

Remember, it is differential equations all the way down. You can use the same Euler method to patiently update each of your terms and then repeatedly loop between them. But recognize there are DEs at multiple levels. Each of the m , n , and h terms are also changing and regulated by a differential equation. They are dependent on voltage, too. For example, $\dot{m} = \alpha_m(V)(1 - m) - \beta_m(V)m$.

Recognize that

- Each of the m, n, and h terms have their own equation of exactly the same form, but with their unique alphas and betas (that is what the subscript means).
- What does the V in parentheses mean?
- When the proteins of the ion channels were finally sequenced (decades later), what do you think was the number of sub-units that the sodium and potassium channels were found to have?

Initial Assumptions

If you allow $t \rightarrow \infty$, then $\frac{dV}{dt} = ?$

- You assume that it goes to zero; that is, you reach steady state. Then you can solve for some of the constants.
- Where do the constants come from?
- They come from experiments, and you use what you are given.
- Assume the following constants - they are set to assume a resting potential of zero (instead of what and why doesn't this matter)?
- These constants also work out to enforce a capacitance of 1.

2. Constants Several different sets of constants are used for this model in published versions. They are all, in one sense the same, but this set amke the programming easier. For the moment, ignore the units and use these numbers.

Table 1: Constant Values to Use for our Hodgkin-Huxley Model

Constant	Value
ena	115
gna	120
ek	-12
gk	36
el	10.6
gl	0.3

Warning These constants are adjusted to make the resting potential 0 and the capacitance 1.0.

3. Alpha and Beta Formulas

$$\alpha_n(V_m) = \frac{0.01(10 - V_m)}{\exp\left(\frac{10 - V_m}{10}\right) - 1}$$

$$\alpha_m(V_m) = \frac{0.1(25 - V_m)}{\exp\left(\frac{25 - V_m}{10}\right) - 1}$$

$$\alpha_h(V_m) = 0.07 \exp\left(\frac{-V_m}{20}\right)$$

$$\beta_n(V_m) = 0.125 \exp\left(\frac{-V_m}{80}\right)$$

$$\beta_m(V_m) = 4 \exp\left(\frac{-V_m}{18}\right)$$

$$\beta_h(V_m) = \frac{1}{\exp\left(\frac{30 - V_m}{10}\right) + 1}$$

4. Demonstrating the Hodgkin-Huxley Model

TODO: reformat code to fit on page

```

1  {-# LANGUAGE GADTs #-}
2
3  module HandH where
4  import Data.Maybe (fromMaybe)
5
6  data SimParameters where
7    SimParameters :: {
8      dt :: Float,
9      maxT :: Float,
10     initT :: Float,
11     startTime :: Float,
12     stopTime :: Float,
13     capacitance :: Float,
14     resistance :: Float,
15     initialVoltage :: Float,
16     injectionCurrent :: Float,
17     tau :: Float
18   } -> SimParameters
19   deriving Show
20
21  data NeuronState where
22    NeuronState :: {
23      vns :: Float
24      , ic  :: Float
25      , neuTime :: Float
26      , mns :: Maybe Float
27      , nns :: Maybe Float
28      , hns :: Maybe Float
29    } -> NeuronState
30    deriving Show
31
32  data NeuronParams where
33    NeuronParams :: {
34      ena :: Float,
35      gna :: Float,
36      ek  :: Float,
37      gk  :: Float,
38      el  :: Float,
39      gl  :: Float
40    } -> NeuronParams

```

```

41     deriving Show
42
43 data NeuronDynamics where
44     NeuronDynamics :: {
45         alphaN :: Float -> Float,
46         alphaM :: Float -> Float,
47         alphaH :: Float -> Float,
48         betaN  :: Float -> Float,
49         betaM  :: Float -> Float,
50         betaH  :: Float -> Float,
51         mdot   :: Float -> Float -> Float,
52         ndot   :: Float -> Float -> Float,
53         hdot   :: Float -> Float -> Float,
54         minf   :: Float -> Float,
55         ninf   :: Float -> Float,
56         hinf   :: Float -> Float
57     } -> NeuronDynamics
58
59 data Neuron where
60     Neuron :: {parameters :: NeuronParams,
61               equations  :: NeuronDynamics
62             } -> Neuron
63
64 dotDynamics :: Float -> Float ->
65             (Float -> Float) ->
66             (Float -> Float) ->
67             Float
68 dotDynamics volt chan alphaf betaf =
69     alphaf volt * (1 - chan) - betaf volt * chan
70
71 dotInfinity :: Float ->
72             (Float -> Float) ->
73             (Float -> Float) ->
74             Float
75 dotInfinity v alphaf betaf = alphaf v / (alphaf v + betaf v)
76
77 pSet1 :: SimParameters
78 pSet1 = SimParameters 0.05 300.0 0.0 100.0 150.0 1.0 2.0 0.0 20.0 (resista
79
80 pSet2 :: SimParameters

```

```

81 pSet2 = SimParameters 0.05 300.0 0.0 10.0 50.0 1.0 2.0 0.0 0.0 (resistance
82
83 healthyParams :: NeuronParams
84 healthyParams = NeuronParams 115 120 (-12) 36 10.6 0.3
85
86 healthyDynamics :: NeuronDynamics
87 healthyDynamics = NeuronDynamics {
88   alphaN = \v -> (0.1-0.01*v)/(exp(1-0.1*v) - 1),
89   alphaM = \v -> (2.5-0.1*v)/(exp(2.5-0.1*v) - 1),
90   alphaH = \v -> 0.07*exp((-v)/20),
91   betaN   = \v -> 0.125 * exp((-v)/80),
92   betaM   = \v -> 4*exp((-v)/18),
93   betaH   = \v -> 1/(exp(3-0.1*v)+1),
94   mdot    = \v c -> dotDynamics v c (alphaM healthyDynamics)
95                                     (betaM healthyDynamics),
96   ndot    = \v c -> dotDynamics v c (alphaN healthyDynamics)
97                                     (betaN healthyDynamics),
98   hdot    = \v c -> dotDynamics v c (alphaH healthyDynamics)
99                                     (betaH healthyDynamics),
100  minf     = \v -> dotInfinity v (alphaM healthyDynamics)
101                                     (betaM healthyDynamics),
102  ninf     = \v -> dotInfinity v (alphaN healthyDynamics)
103                                     (betaN healthyDynamics),
104  hinf     = \v -> dotInfinity v (alphaH healthyDynamics)
105                                     (betaH healthyDynamics)
106  }
107
108 healthyNeuron :: Neuron
109 healthyNeuron = Neuron healthyParams healthyDynamics
110
111 initialState :: NeuronState
112 initialState = NeuronState (initialVoltage pSet1) (injectionCurrent pSet1)
113
114 updNeuron :: Neuron -> SimParameters -> NeuronState -> NeuronState
115 updNeuron neuin spin nsin =
116   let eqneuin = equations neuin
117       voltagein = vns nsin
118       nparamsin = parameters neuin
119       outi = ic nsin
120       mnew = fromMaybe 0.0 (case mns nsin of

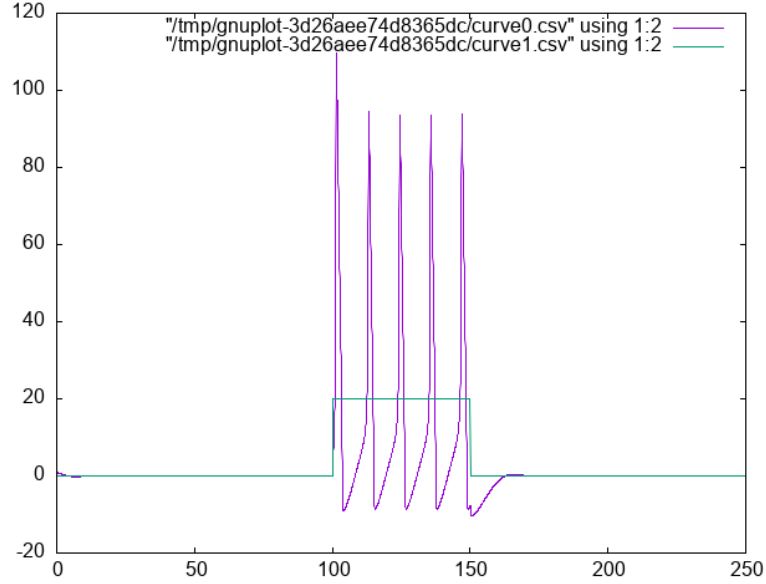
```

```

121             Nothing -> Just $ minf eqneuin voltagein
122             Just x   -> Just $ x + mdot eqneuin voltagein
123     nnew = fromMaybe 0.0 (case nns nsin of
124             Nothing -> Just $ ninf eqneuin voltagein
125             Just x   -> Just $ x + ndot eqneuin voltagein
126     hnew = fromMaybe 0.0 (case hns nsin of
127             Nothing -> Just $ hinf eqneuin voltagein
128             Just x   -> Just $ x + hdot eqneuin voltagein
129     dvdt = outi -
130         (gna nparamsin * mnew ^ 3 * hnew *
131         (voltagein - ena nparamsin) +
132         gk nparamsin * nnew ^ 4 *
133         (voltagein - ek nparamsin) +
134         gl nparamsin *
135         (voltagein - el nparamsin))
136     in
137     nsin {vns = voltagein + dvdt * dt spin
138           , ic = if (neuTime nsin < stopTime spin) &&
139                   (neuTime nsin > startTime spin)
140               then injectionCurrent spin
141               else 0.0
142           , neuTime = neuTime nsin + dt spin
143           , mns = Just mnew
144           , nns = Just nnew
145           , hns = Just hnew
146           }

:load ./code/HandH.hs
let myData = take 5000 $ iterate (updNeuron healthyNeuron pSet1 ) initialState
vs = map vns myData
cs = map ic myData
ts = map neuTime myData
tv = zip ts vs
tc = zip ts cs
plotPaths [PNG "./images/handh.png"] [tv,tc]

```



5. Hodgkin and Huxley Homework

Your homework will require you to write the code for a Hodgkin-Huxley Neuron and then modify it. You will provide at least two plots in addition to your code. You will need to adapt the model to fit this paper. These authors analyzed a series of inherited channelopathies (where ion channels are altered due to mutation) via simulation. In these conditions empirical measurements had been made on actual patients. The authors adapted the Hodgkin and Huxley model to permit them to explore the effects of such altered conductance on neuronal excitability. For this home work you will need to adapt the n equation to be as follows:

$$\frac{dn}{dt} = \gamma_{\tau}(\gamma_{\alpha}\alpha_n(v - \Delta V)(1 - n) - \gamma_{\beta}\beta_n(v - \Delta V)n)$$

Then you must select one of the sets of γ from their Table 1 and plot the neuronal activity.

To start implement the new n channel and set all the γ s to 1. This should work exactly like the old model. Generate a plot showing that it does. For the second plot generate the same plot with your altered γ values.

3 What is computation and what is its role in cognitive science?

Chapter 1 of Physical Computation in Cognitive Science by Nir Fresco

3.1 What do you think?

1. Computational modeling of cognition and neural systems is/is-not the same as programming.
2. What else could computational modeling be?
3. What are we losing when we treat our computational model as a programmatic black box?
4. Is the computation that is cognitive the same as the computation that takes place in digital computers?
5. If so, why is there more than one programming language?
6. Might our choice of programming language based on features actually influence what we can conceive for a computational model?
7. What does it mean for something to be a model?
8. Is simulation good enough?
9. Is prediction good enough?
10. Do we want a model to *explain* something? And then of course what constitutes explanation?

3.2 What do we need to know?

1. We need to know the facts of the matter, neurally and cognitively.
2. We need to have a clear question.
3. We need to have a theoretical commitment that leads to clear predictions, and guides our evaluation of what to do.
4. Which of course means we need to know what a theory is and how to build one.

5. We need to know the facts of the matter mathematically and programmatically
 - (a) We need to be able to read the maths.
 - (b) We need to be conversant in symbols and notations.
 - (c) We need to know how to express in symbols what we can say in words.
 - (d) We need to know how to express in code what we can write in symbols.

We need to be concerned with all the levels: mind and brain, theory, math, and code. We can abbreviate these as MTMC: mind $\rightarrow$ theory $\rightarrow$ math $\rightarrow$ code; the arrows indicating an ordering on our development.

3.3 Computing in Minds and Computers

One definition of whether something is computable is whether there exists a Turing Machine that can compute it. Although most of us learned the name "Turing" in the context of whether a computer has human intelligence, the so-called *Turing test*, the model of Turing computability emerged from thinking about humans computing

3.4 What is a Turing machine? Some Background and Details

Turing machines are quite simple implements. While one can build a physical Turing machine, the more usual sense of the term is for a hypothetical computer that is comprised of:

1. a finite alphabet,
2. a finite set of states,
3. the capacity to read and write to a single location in memory, and the ability to adjust the memory location immediately left or right or to make no move at all, and
4. a set of instructions (or "machine table") that translates the combination of the current state and the current symbol to a new state and one of the acceptable actions.

3.4.1 Classroom Activity

Let's develop pseudo-code for the implementing a Turing Machine

3.5 The Lambda Calculus

Another model of computation is the *lambda calculus*.

The lambda calculus was developed as a theory of functions. John McCarthy invented the computer programming language Lisp as a theoretical exercise for working on the theory of computable functions. He felt the Turing machine was too mechanical and too awkward for this sort of theoretical work. He wanted a better tool; a better metaphor. He adopted the lambda of the lambda calculus and the `eval` function to take in lisp programs and execute them. Later one of his collaborators observed that it was relatively straightforward to implement this theoretical language as a real programming language. A bit more of the history this development can be found here. To get more details see.

3.5.1 Some Details and Interesting Facts about the Lambda Calculus

1. There is not just one lambda calculus.
2. To have the lambda calculus you need to specify your algebra. What are the rules for reducing things (i.e. your computations), what are the allowed symbols, and what do things mean? A "lambda calculus" is a way of handling "lambda expressions."¹

3.5.2 Into the Weeds

Terms are either *simple variables* e.g. x or y or *composite terms* like $\lambda v . t_1$. Having two terms next to each other $t_1 t_2$ means "apply" t_1 to t_2 . The meaning of a term like $\lambda v . t_1$ is the value returned by the lambda abstraction. The meaning part is sometimes designated in writing by formulas using arrows, such as $t_1 \rightarrow t_2$.²

Functions have the form $\lambda \langle \text{name} \rangle . \langle \text{body} \rangle$. Note the "dot". This separates the name from the body of expressions that it names. $\langle \text{body} \rangle$ is also an expression (note the recursion that is built in).

Some "axioms" you supply, or that Alonzo Church did are:

What are recursive functions?

¹see David Marr's **Vision**.

²The limits of my language means the limits of my world. – Ludwig Wittgenstein (proposition 5.6 Tractatus Logico-Philosophicus)

- Beta reduction apply the function, e.g. $(\lambda x.M) N = M[x:N]$ this latter is just a way of saying that you will substitute the value "N" everytime you see an "x" in whatever expression "M" is.
- Alpha conversion essentially you are just renaming variables
- Eta reduction Is a way of getting rid of things that don't change anything. If for example $f(x) = g(x)$ then you could just save yourself some ink and write: $f = g$. A more technical way of saying things, that we will not go into, is that you can free yourself of free variables, that is if there is no "x" in "f" then $f = \lambda x.(fx)$.

Your goal is often the **normal** form. A lambda term that cannot be reduced anymore. Think of it like simplifying an equation. It helps you get to the essentials, and can make it easier to compare things to see when they are the same.

The equivalent of the halting problem for the Turing machine is the reaching of a *normal form* in the lambda calculus.

3.5.3 Some Opportunities for Practice

- Write the lambda expression for the identity function?
 - But first we better decide what the identity function is?
- Apply the identity function to itself.
- What is the identity function in whatever language you are hoping to use?
- An interesting lambda expression is the "self-application" expression: $\lambda s.(s\ s)$.
 - With pencil and paper try to:
 - * apply this to the identity,
 - * apply the identity to the self-application,
 - * apply the self-application to itself. What is its termination status?

3.5.4 What are the differences?

We have seen Turing Machines and a Lambda Calculus. They look very different. Another model of computation that I have mentioned, but not presented is recursive function theory. It looks different still.

What does it mean for our idea that the mind is a "computer" if there are three different models of computation?

Always question assumptions: Church-Turing Conjecture .

1. Are Turing machines /digital?
2. Is this an important distinction?
3. Does this mean that analog computations are omitted?
4. Would an analog paradigm be a better match to modeling mental activity?

3.6 Classical Computational Theory of Mind

Does a computational theory of mind mean that minds are multiply realizable ?

The classical computational theory of mind says that in all important ways the mind is like a Turing machine. If we want to model (or emulate) functions of mind one way would be to build a Turing machine. However, this can be a practical challenge, and one can question the insight gained from this approach. Still given the theoretical and practical prominence given to Turing computability it behooves us to know what a Turing machine truly is.

For something to be computable the Turing machine that computes it must halt. So, we would like to know before we run our machine whether it will halt or not. Unfortunately, the answer to the halting problem is that we cannot know in advance, and in general, whether a Turing machine will halt. Computability is therefore *undecidable*.

3.7 Programming a Turing Machine: the Busy Beaver

In my experience the only way to really come to grips with what a Turing machine is is to program one yourself. For this exercise we will program a Turing machine to solve an elementary version of the Busy Beaver problem.

This is a famous problem, because it is easy to state and woefully difficult to answer. In fact it is uncomputable.

The following is a slightly formatted version of the Wikipedia description of a Turing machine.

1. A Turing machine has n "operational" states plus a Halt state, where n is a positive integer, and one of the n states is distinguished as the starting state.
2. The machine uses a single two-way infinite (or unbounded) tape.
3. The tape alphabet is $\{0, 1\}$, with 0 serving as the blank symbol.
4. The machine's transition function takes two inputs:
 - (a) the current non-Halt state,
 - (b) the symbol in the current tape cell,
 - (c) and produces three outputs:
 - i. a symbol to write over the symbol in the current tape cell (it may be the same symbol as the symbol overwritten),
 - ii. a direction to move (left or right@";" that is, shift to the tape cell one place to the left or right of the current cell), and
 - iii. a state to transition into (which may be the Halt state).

We will use it to guide us to write a simple instance that computes a solution to the Busy Beaver problem. A n -state Turing machine has $(4n + 4)2n$ states. The formula is $(\text{symbols} \times \text{directions} \times (\text{states} + 1))(\text{symbols} \times \text{states})$. The transition function (how to figure out where to go next may be seen as a finite look-up table. Each row of the table is a 5-tuple: (current state, current symbol, symbol to write, direction of shift, next state). Our goal with the Busy Beaver problem is to run our machine to produce as long a series of uninterrupted ones as we can and then **halt**.

What it means to "run" a Turing machine is to start in the starting state with the current tape cell being any cell of a blank (all-0) "tape", and then iterate the transition function. If the Halt state is entered then the number of 1s remaining on the tape is called the machine's score. Different transition rules will give us different outputs, so we can score our machine based on its performance.

To restate in a more general way, the n -state busy beaver (BB- n) game is a contest to find an n -state Turing machine having the largest possible score — the largest number of 1s on its tape after halting. A machine that

attains the largest possible score among all n -state Turing machines is called an n -state busy beaver, and a machine whose score is merely the highest so far attained (perhaps not the largest possible) is called a champion n -state machine.

3.7.1 A Busy Beaver Warm-Up

A simple version of the Busy Beaver problem, and one you can do by hand with pencil and paper, is the $n=2$ version. Create a Turing Machine with the following transition rules:

Here is the transition table for our machine:

Machine Tuple In	Machine Tuple Out
a0	b1r
a1	b1l
b0	a1l
b1	h1r

3.7.2 Busy Beaver Problem Demo Code

```

1  module BusyBeaver where
2
3  import Data.Set (Set,fromList)
4  import qualified Data.Map.Lazy as Map
5  import Data.Map.Lazy (Map)
6
7  type TMState = Char
8  type Position = Int
9  data BinNum = Zero | One deriving (Show, Eq, Ord, Enum)
10 type TMTape = Map Int BinNum
11
12 data TuringMachine = TM
13   { state :: TMState
14     , tape  :: TMTape
15     , headLoc :: Int} deriving Show
16
17 testStates :: Set TMState
18 testStates = fromList ['a','b','h']
19
20 initialTape :: Map Int BinNum
21 initialTape = Map.empty

```

```

22
23 defaultTapeCell :: BinNum
24 defaultTapeCell = Zero
25
26 initialState :: TMState
27 initialState = 'a'
28
29 initialTM :: TuringMachine
30 initialTM = TM initialState initialTape 0
31
32 readFromTape :: TMtape -> Int -> BinNum
33 readFromTape t i =
34     Map.findWithDefault Zero i t
35
36 writeToTape :: BinNum -> Int -> TMtape -> TMtape
37 writeToTape bn hl = Map.insert hl bn
38
39
40 updTM :: TuringMachine -> TuringMachine
41 updTM tm@(TM st t hl) =
42     let bn = readFromTape t hl in
43     case st of
44         'a' -> case bn of
45             Zero -> tm {state = 'b'
46                 , tape = writeToTape One hl t
47                 , headLoc = hl + 1}
48             One -> tm {state = 'b'
49                 , tape = writeToTape One hl t
50                 , headLoc = hl - 1}
51         'b' -> case bn of
52             Zero -> tm {state = 'a'
53                 , tape = writeToTape One hl t
54                 , headLoc = hl - 1}
55             One -> tm {state = 'h'
56                 , tape = writeToTape One hl t
57                 , headLoc = hl }
58         'h' -> tm
59         _ -> error "Entered incorrect state variable"

:load ./code/BusyBeaver.hs

```

```
let d = iterate updTM initialTM
tape $ d !! 7
```

```
fromList [(-2,One),(-1,One),(0,One),(1,One)]
```

1. Busy Beaver Homework See Dropbox on Learn
You might find the racket source useful for hints.

4 Theory

These are just place holders for now.

I have been wondering about the concept of explanation. I don't know if there is an agreed upon definition in this context. How do we know that an observation is "explained" other than that it is predicted by a theory? Or is the mathematics the same, and is explanation synonymous with post-diction? – Joachim Vanderkerckhove

Is the ability to predict the tides a theory of the tides? Is it an explanation of the tides?

Formalizing Verbal Theories. A tutorial by dialogue.

Forms of explanation and understanding for neuroscience and artificial intelligence

Models are Stupid, and We Need More of Them

Productive Explanation: A framework for evaluating explanations in psychological science

Theory Construction Methodology: A practical framework for building theories in psychology

5 Math and Code

We will group these two by subdomains to keep the connections fresh.